# Atelier ##1 — Explorer React Native avec Expo

**Durée estimée :** 20 minutes

## 🎯 Objectifs

Dans cet atelier, vous allez découvrir l'environnement React Native avec Expo en lançant votre première application sur Expo Go et en explorant la structure générée automatiquement.

- Lancer un projet Expo fraîchement généré sur Expo Go
- Explorer la structure d'un projet React Native avec Expo
- Découvrir les Core Components et quelques APIs de base de React Native
- Observer les différences entre React (DOM) et React Native

À la fin de l'atelier, vous aurez réalisé :

- Le lancement d'une application Expo sur votre smartphone (via Expo Go)
- Une première modification visible dans l'application
- Une exploration guidée des fichiers clés du projet
- L'utilisation de plusieurs Core Components ( `View` , `Text` , `Image` , `Button` )

# Étape 1 — Lancer le projet Expo sur Expo Go

### Objectifs

- Comprendre comment démarrer un projet Expo
- Connecter son smartphone à l'environnement de développement
- Observer la Developer Experience (DX) proposée par Expo

### Comment réaliser cette étape ?

1. Lancez le serveur de développement :

```
npx expo start
```

2. Une interface s'ouvre dans le navigateur avec un QR Code.
3. Installez l'application **Expo Go** sur votre smartphone (si ce n'est pas déjà fait).
4. Scannez le QR Code avec :
   - L'appareil photo (iPhone)
   - L'application Expo Go (Android)
5. Attendez que le bundle JS soit chargé sur votre téléphone.

### Attendus

- L'application par défaut s'affiche sur votre smartphone.
- Vous constatez qu'aucun build natif n'a été nécessaire.
- Vous observez la rapidité du rechargement.

💡 **Questions à se poser :**

- Que se passe-t-il réellement entre votre ordinateur et votre téléphone ?
- Où le code JavaScript est-il exécuté ?
- Pourquoi n'a-t-on pas eu besoin d'Android Studio ou Xcode ?

# Étape 2 — Explorer la structure du projet

### Objectifs

- Identifier les fichiers et dossiers importants
- Comprendre l'organisation d'un projet React Native avec Expo
- Comparer avec un projet React (DOM)

### Comment réaliser cette étape ?

1. Ouvrez le projet dans votre éditeur (VS Code recommandé).
2. Observez les fichiers suivants :
   - `App.tsx`
   - `package.json`
   - `app.json`
   - Le dossier `assets`
3. Ouvrez `App.tsx` .

4. Repérez :
    - Le composant principal exporté par défaut
    - Les imports provenant de `react-native`
    - Le JSX retourné
5. Comparez mentalement avec un projet React (DOM) :
    - Où est le `index.html` ?
    - Où est le `ReactDOM.createRoot` ?

### Attendus

- Vous identifiez que `App.tsx` est le point d'entrée principal.
- Vous comprenez qu'il n'y a pas de DOM.
- Vous remarquez que les composants viennent de `react-native` et non de `react` .

💡 **Questions à se poser :**

- Si on n'utilise pas le DOM, qu'est-ce qui affiche réellement l'interface ?
- Que contient `app.json` ? À quoi sert-il ?

# Étape 3 — Découvrir les Core Components

### Objectifs

- Manipuler les composants de base de React Native
- Observer les différences avec le HTML
- Comprendre le rôle de `View` et `Text`

### Comment réaliser cette étape ?

1. Dans `App.tsx` , remplacez le contenu du `return` par :

```
import { View, Text, Button, Image } from "react-native";
import { useState } from "react";

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>Bienvenue dans React Native 🚀</Text>
      <Text>Compteur : {count}</Text>
      <Button title="Incrémenter" onPress={() => setCount(count + 1)} />
      <Image
        source={{ uri: "https://reactnative.dev/img/tiny_logo.png" }}
        style={{ width: 50, height: 50, marginTop: 20 }}
      />
    </View>
  );
}
```

2. Sauvegardez le fichier.
3. Observez le rechargement automatique sur votre téléphone.

## Attendus

- Le texte s'affiche correctement.
- Le bouton incrémente le compteur.
- L'image apparaît.
- Vous observez le fonctionnement du **Fast Refresh**.

💡 **Questions à se poser :**

- Pourquoi utilise-t-on `View` au lieu de `div` ?
- Pourquoi le style ressemble-t-il à du CSS-in-JS ?
- Où sont réellement rendus ces composants ?

# Étape 4 — Explorer une API simple : `Alert`

## Objectifs

- Découvrir qu'il existe des APIs natives accessibles en JavaScript
- Comprendre que React Native fait le lien avec le système natif

## Comment réaliser cette étape ?

1. Modifiez votre import :

```
import { View, Text, Button, Image, Alert } from "react-native";
```

2. Modifiez le bouton :

```
<Button
  title="Afficher une alerte"
  onPress={() => Alert.alert("Hello 👋", "Ceci est une alerte native !")}
/>
```

3. Sauvegardez et testez.

## Attendus

- Une alerte native s'affiche sur votre téléphone.
- L'alerte a l'apparence du système (iOS ou Android).

💡 **Questions à se poser :**

- Pourquoi le design de l'alerte change-t-il selon le téléphone ?
- Où se trouve le code natif correspondant ?
- Comment JavaScript peut-il déclencher une fonctionnalité native ?

# Fin de l'atelier

🎉 Félicitations !

En 20 minutes, vous avez :

- Lancé une application Expo sur un appareil réel
- Exploré la structure d’un projet React Native
- Manipulé plusieurs Core Components
- Utilisé une API native simple
- Observé la Developer Experience proposée par Expo

👉 Cet atelier était volontairement exploratoire.
Gardez en tête les nombreuses questions soulevées : elles serviront de point d’entrée pour comprendre plus en profondeur :

- L’architecture de React Native
- Le bridge JavaScript ↔ natif
- Le rôle exact d’Expo
- Les différences fondamentales avec React (DOM)

Dans les prochains ateliers, nous approfondirons chacun de ces points.